

Lab 11 Test Report: Symbol Table Manager

Abdul Rehman

May 2026

Objective

This lab implemented a hash-based symbol table manager for a Pascal subset. The main goals were to support identifier insertion, lookup, deletion, nested scopes, duplicate declaration detection, undeclared name detection, and parser integration.

Implementation Summary

The symbol table is implemented in `symbol_table.py` using separate chaining with a prime table size of 211. Each scope is represented as an independent hash table with a parent pointer. The parser integration is connected to the recursive descent parser from Lab 09, while the semantic driver `pascal_semantic_simulator.py` is used for focused testing.

Test Cases and Results

1. Task 1: single-scope operations

- 10 inserts, 5 lookups, and 3 deletes were executed successfully using `task1_driver.py`.
- Lookups included misses, and deletes correctly removed entries from the current scope.

2. Valid nested scope program

```
python lab_11/pascal_semantic_simulator.py lab_11/test_cases/valid_nested.pas
```

Expected: no semantic errors. Actual: passes with nested scope dumps, and all lookups resolve correctly.

3. Duplicate declaration

```
python lab_11/pascal_semantic_simulator.py lab_11/test_cases/duplicate_decl.pas
```

Expected: duplicate declaration error for x. Actual:

```
ERROR line 4: duplicate declaration 'x'
```

4. Undeclared variable use

```
python lab_11/pascal_semantic_simulator.py lab_11/test_cases/undeclared_use.pas
```

Expected: undeclared variable error for a. Actual:

```
ERROR line 5: undeclared variable 'a'
```

5. Shadowing in nested scope

```
python lab_11/pascal_semantic_simulator.py lab_11/test_cases/shadowing_valid.pas
```

Expected: inner `x` should shadow outer `x` inside the nested block. Actual: the inner declaration is used inside scope 2, and the outer declaration is restored after scope exit.

Observed Output Summary

The output from the tested programs showed the following behavior:

- Scope entry and exit messages were printed correctly.
- Each scope was dumped in a formatted table at exit.
- Duplicate declaration and undeclared variable errors were reported with line numbers.
- Shadowing worked as expected, with lookup returning the nearest declaration.

Conclusion

The lab successfully demonstrates the use of a symbol table in a compiler front-end. The implementation meets the required operations for single-scope hashing, nested scope management, error detection for duplicate and undeclared names, and debug printing. The recursive descent parser integration provides a realistic path for semantic analysis in the final compiler project.